

OOP Conclusion Ver2

- 1. Chapter 9: Objects and Classes (对象和类)
 - 1.1. 引言与目标 (Introduction & Objectives)
 - 1.2. 面向对象编程核心概念 (Core OOP Concepts)
 - 1.3. 对象创建与访问 (Object Creation & Access)
 - 1.4. 数据字段与变量 (Data Fields & Variables)
 - 1.5. 内存管理与特殊关键字 (Memory Management & Special Keywords)
 - 1.6. Java 库类示例 (Java Library Classes Examples)
 - 1.7. 实例与静态成员 (Instance vs. Static Members)
 - 1.8. 可见性与封装 (Visibility & Encapsulation)
- 2. Chapter 10: Thinking in Objects (面向对象思考)
 - 2.1. 引言与目标 (Introduction & Objectives)
 - 2.2. 类抽象与封装 (Class Abstraction & Encapsulation)
 - 2.3. 面向对象设计原则 (Object-Oriented Design Principles)
 - 2.4. 类关系 (Class Relationships)
 - 2.5. 包装类与大数处理 (Wrapper Classes & Large Number Handling)
 - 2.6. 字符串处理 (String Handling)
- 3. Chapter 11: Inheritance and Polymorphism (继承与多态)
 - 3.1. 引言与目标 (Introduction & Objectives)
 - 3.2. 继承基础 (Inheritance Basics)
 - 3.3. 构造函数与继承 (Constructors & Inheritance)
 - 3.4. 方法重写与重载 (Method Overriding & Overloading)
 - 3.5. 多态与动态绑定 (Polymorphism & Dynamic Binding)
 - 3.6. 类型转换与操作符 (Casting & Operators)
 - 3.7. 集合框架中的应用 (Applications in Collections Framework)
 - 3.8. 访问修饰符与 `final` (Access Modifiers & `final`)
- 4. Chapter 12: Exception Handling and Text IO (异常处理和文本I/O)
 - 4.1. 引言与目标 (Introduction & Objectives)
 - 4.2. 异常处理基础 (Exception Handling Basics)
 - 4.3. 异常的声明、抛出与捕获 (Declaring, Throwing, and Catching Exceptions)
 - 4.4. 异常处理的实践指导 (Practical Guidance for Exception Handling)
 - 4.5. 文件I/O基础 (File I/O Basics)
 - 4.6. 二进制I/O与对象序列化 (Binary I/O & Object Serialization)

- 5. Chapter 13: Abstract Classes and Interfaces (抽象类和接口)
 - 5.1. 引言与目标 (Introduction & Objectives)
 - 5.2. 抽象类 (Abstract Classes)
 - 5.3. 抽象类应用示例 (Abstract Class Application Examples)
 - 5.4. 接口 (Interfaces)
 - 5.5. 接口应用示例 (Interface Application Examples)
 - 5.6. 接口与抽象类的对比 (Interfaces vs. Abstract Classes)
 - 5.7. 类设计指南 (Class Design Guidelines)
- 6. Chapter 14: JavaFX Basics (JavaFX 基础)
 - 6.1. 引言与目标 (Introduction & Objectives)
 - 6.2. JavaFX 概述 (JavaFX Overview)
 - 6.3. JavaFX UI 元素 (JavaFX UI Elements)
 - 6.4. 布局管理 (Layout Management)
 - 6.5. 属性与样式 (Properties & Styling)
 - 6.6. 图像与多媒体 (Images & Multimedia)
- 7. Chapter 15: Event-Driven Programming and Animations (事件驱动编程和动画)
 - 7.1. 引言与目标 (Introduction & Objectives)
 - 7.2. 事件驱动编程核心 (Core Event-Driven Programming)
 - 7.3. 事件类与信息 (Event Classes & Information)
 - 7.4. 事件处理方式 (Event Handling Approaches)
 - 7.5. JavaFX 动画 (JavaFX Animations)
- 8. Chapter 16: JavaFX UI Controls and Multimedia (JavaFX UI 控件和多媒体)
 - 8.1. 引言与目标 (Introduction & Objectives)
 - 8.2. UI 控件基础 (UI Control Basics)
 - 8.3. 文本输入与选择控件 (Text Input & Selection Controls)
 - 8.4. 数值选择控件 (Numeric Selection Controls)
 - 8.5. 多媒体与案例研究 (Multimedia & Case Studies)
- 9. Chapter 17: Binary I/O (二进制 I/O)
 - 9.1. 引言与目标 (Introduction & Objectives)
 - 9.2. Java I/O 概述 (Java I/O Overview)
 - 9.3. 二进制 I/O 流 (Binary I/O Streams)
 - 9.4. 对象序列化 (Object Serialization)
 - 9.5. 随机访问文件 (Random Access Files)
- 10. Chapter 19: Generics (泛型)
 - 10.1. 引言与目标 (Introduction & Objectives)
 - 10.2. 泛型基础 (Generics Basics)
 - 10.3. 泛型类与接口 (Generic Classes & Interfaces)
 - 10.4. 泛型方法与通配符 (Generic Methods & Wildcards)

- 10.5. 类型擦除与泛型限制 (Type Erasure & Generics Restrictions)
- 11. Chapter 20: Lists, Stacks, Queues, and Priority Queues (列表、栈、队列和优先队列)
 - 11.1. 引言与目标 (Introduction & Objectives)
 - 11.2. 数据结构与集合框架 (Data Structures & Collections Framework)
 - 11.3. 列表 (Lists)
 - 11.4. 比较器 (Comparators)
 - 11.5. 栈 (Stacks)
 - 11.6. 队列与优先队列 (Queues & Priority Queues)

1. Chapter 9: Objects and Classes (对象和类)

1.1. 引言与目标 (Introduction & Objectives)

传统编程方法不足以开发复杂 GUI 和大型软件系统，OOP 提供了更好的解决方案。本章目标包括：描述对象和类；使用 UML；定义类、创建对象和使用构造函数；通过对象引用变量访问成员；区分对象引用与基本类型；使用 `Date`、`Random`、`Point2D` 等 Java 库类；区分实例与静态成员；定义私有字段及 `get / set` 方法；封装数据；开发带对象参数的方法；在数组中处理对象；创建不可变对象；确定变量作用域；使用 `this` 关键字。

1.2. 面向对象编程核心概念 (Core OOP Concepts)

- **面向对象编程 (OOP)** 涉及使用对象进行编程。对象是现实世界中可识别的实体，具有唯一的身份、状态（数据字段/属性）和行为（方法）。
- **对象 (Objects)** 既有状态也有行为。状态定义对象，行为定义对象做什么。类是创建对象的模板，对象是类的实例。
- **类 (Classes)** 是定义相同类型对象的构造。Java 类使用变量定义数据字段，使用方法定义行为。构造函数是特殊方法，用于从类构造对象。
- **UML 类图 (UML Class Diagram)** 用于图形化表示类和对象。一个类在 UML 中通常分为三部分：类名、数据字段和方法。对象实例表示对象名和数据字段值。

1.3. 对象创建与访问 (Object Creation & Access)

- **构造函数 (Constructors)** 是用于构造对象的特殊方法。
 - 无参构造函数是没有参数的构造函数。
 - 构造函数必须与类同名，且没有返回类型（包括 `void`）。
 - 使用 `new` 运算符调用构造函数来初始化对象。
 - 如果类中未显式定义任何构造函数，Java 会自动提供一个带有空体的无参默认构造函数。

- **声明对象引用变量 (Declaring Object Reference Variables)** 语法: `ClassName objectRefVar;`
- **可以一步声明/创建对象 (Declaring/Creating Objects in a Single Step):** `ClassName objectRefVar = new ClassName();`
- **访问对象成员 (Accessing Object's Members)** 使用点运算符 (`.`)。
 - 引用数据: `objectRefVar.data` (例如 `myCircle.radius`)。
 - 调用方法: `objectRefVar.methodName(arguments)` (例如 `myCircle.getArea()`)。

1.4. 数据字段与变量 (Data Fields & Variables)

- **引用数据字段 (Reference Data Fields)** 可以是引用类型 (如 `String`)。如果没有引用任何对象, 会持有特殊值 `null` 。
- **数据字段的默认值 (Default Value for a Data Field):** 引用类型为 `null` , 数值类型为 `0` , 布尔类型为 `false` , 字符类型为 `'\u0000'` 。 **注意:** Java 不会为方法内部的局部变量分配默认值, 它们必须在使用前显式初始化。
- **基本数据类型变量与对象类型变量的区别 (Differences between Variables of Primitive Data Types and Object Types):** 基本类型变量直接存储值, 对象类型变量存储的是对象的引用 (内存地址)。
- **复制基本数据类型变量和对象类型变量 (Copying Variables of Primitive Data Types and Object Types):** 基本类型赋值复制值本身, 两个变量独立; 对象类型赋值复制引用, 两个变量指向同一个对象。

1.5. 内存管理与特殊关键字 (Memory Management & Special Keywords)

- **垃圾回收 (Garbage Collection):** 当对象不再被任何引用变量指向时, 它成为“垃圾”, JVM 会自动回收其占用的内存空间。可以显式地将 `null` 赋给引用变量以帮助 JVM 识别不再需要的对象。
- **变量作用域 (Scope of Variables):** 实例变量和静态变量作用域是整个类; 局部变量作用域从声明处到代码块结束, 必须在使用前显式初始化。
- **this 关键字 (The this Keyword):** `this` 引用对象本身。常见用法包括引用类中被隐藏的数据字段, 以及允许构造函数调用同类的另一个构造函数 (构造函数链)。

1.6. Java 库类示例 (Java Library Classes Examples)

- `java.util.Date` : 用于处理日期和时间。
- `java.util.Random` : 用于生成伪随机数序列。
- `javafx.geometry.Point2D` : 用于表示二维平面上的点。

1.7. 实例与静态成员 (Instance vs. Static Members)

- **实例变量和方法**：属于特定对象（实例），通过对象调用。
- **静态变量和方法**：由类的所有实例共享，不绑定到特定对象，通过类名调用。静态变量和方法在 UML 中通常以下划线表示。
- **静态常量**：使用 `final static` 声明，由所有实例共享。

1.8. 可见性与封装 (Visibility & Encapsulation)

- **可见性修饰符 (Visibility Modifiers)**：`public` (任何地方可见)、`private` (只能在类内访问)、默认 (同一包内访问)、`protected` (同一包内及子类访问)。子类不能削弱超类中方法的可见性。
- **为什么数据字段应该是私有的？ (Why Data Fields Should Be private?)**：保护数据；使代码易于维护（封装）。封装数据字段使类易于维护。
- **不可变对象和类 (Immutable Objects and Classes)**：不可变对象内容一旦创建就不能改变。通常通过将所有数据字段设置为 `private` 且不提供 `set` 方法来实现。即使没有修改器，如果返回了对其内部可变对象的引用，类仍可能不是不可变的。

2. Chapter 10: Thinking in Objects (面向对象思考)

2.1. 引言与目标 (Introduction & Objectives)

本章旨在演示如何使用面向对象范式解决问题，以利用其灵活性和模块化，构建可重用软件。目标包括：应用类抽象；探索过程式与面向对象范式差异；发现类间关系；设计面向对象程序；使用包装类处理基本值；简化基本类型与包装类自动转换；使用 `BigInteger` 和 `BigDecimal` 处理大数；使用 `String` 处理不可变字符串；以及使用 `StringBuilder` 和 `StringBuffer` 处理可变字符串。

2.2. 类抽象与封装 (Class Abstraction & Encapsulation)

类抽象：将类实现与类使用分离。类创建者提供类描述（即契约），用户无需了解具体实现细节。

封装：实现细节对用户隐藏。

类契约：由公共方法签名和公共常量组成，客户端通过契约使用类。

2.3. 面向对象设计原则 (Object-Oriented Design Principles)

- **内聚性 (Coherence)**：一个类应描述一个单一实体，所有操作应逻辑地支持一个内聚目的。
- **分离职责 (Separating Responsibilities)**：职责过多的实体可拆分为多个类。例如，`String`、`StringBuilder`、`StringBuffer` 分别处理不可变、可变（非同步）和可变（同步）字符串。

- **重用性**：设计类时应考虑其在不同组合、顺序和环境下的重用性。属性应可按任意顺序设置，方法应独立于其调用顺序。
- 提供**公共无参构造函数**，并尽可能重写 `Object` 类中的 `equals` 和 `toString` 方法。
- 遵循标准 Java 编程风格和命名约定。选择信息丰富的名称。始终将数据声明放在构造函数之前，构造函数放在方法之前。始终提供构造函数并初始化变量，以避免编程错误。
- **可见性修饰符 (Visibility Modifiers)**： `private` (隐藏数据，通过 `get/set` 访问)； `protected` (供扩展者使用)； `public` (供用户使用)。扩展者可以提高实例方法的可见性，但不应以违反契约的方式改变实现。
- **static 修饰符 (The static Modifier)**：由类的所有实例共享的属性应声明为 `static` 属性。

2.4. 类关系 (Class Relationships)

- **关联 (Association)**：两个类之间的通用二元关系（例如，学生“选修”课程，教师“教授”课程）。
- **聚合 (Aggregation)**：一种“has-a”关系，表示一种弱所有权（例如，学生“拥有”姓名，但姓名可以独立存在）。通常表示为聚合类中的数据字段。
- **组合 (Composition)**：聚合的特殊情况，表示一种强所有权，部分不能独立于整体存在（例如，学生“拥有”地址，但地址可能只存在于学生对象中）。许多教材不区分聚合和组合，都统称为组合。
- **同类间的聚合 (Aggregation Between Same Class)**：聚合可以存在于同一类的对象之间（例如，一个人可以有一个主管，而主管也是人）。当一个人有多个主管时，可以使用数组表示主管列表。
- **继承 (Inheritance)**：一种“is-a”关系（在 Chapter 11 中详细讨论）。

2.5. 包装类与大数处理 (Wrapper Classes & Large Number Handling)

- **包装类 (Wrapper Classes)**：Java 为每种基本数据类型提供了对应的包装类（`Byte`，`Short`，`Integer`，`Long`，`Float`，`Double`，`Character`，`Boolean`）。它们是不可变的，没有无参构造函数。
 - 提供构造函数、类常量（`MAX_VALUE`，`MIN_VALUE`）、转换方法（如 `intValue()`）、静态 `valueOf` 方法（从字符串创建对象）和字符串解析方法（如 `parseInt()`）。
 - JDK 1.5 引入了**自动装箱 (autoboxing)** 和**自动拆箱 (unboxing)**，允许基本类型和包装类之间自动转换，简化了编程，且无需显式类型转换。
- **`BigInteger` 和 `BigDecimal`**：用于处理非常大的整数或高精度浮点值。它们都在 `java.math` 包中，是不可变的，扩展了 `Number` 类并实现了 `Comparable` 接口。

2.6. 字符串处理 (String Handling)

- **`String` 类 (The String Class)**：用于处理字符串。字符串是不可变的，其内容一旦创建就不能改变。重新赋值意味着变量指向新的 `String` 对象。
- **内部化字符串 (Interned Strings)**：为提高效率和节省内存，JVM 对字符串字面量使用唯一的实例。

- **替换和分割字符串 (Replacing and Splitting Strings)**: `String` 类提供 `replace()`、`replaceFirst()`、`replaceAll()` 和 `split()` 等方法。还可以使用正则表达式进行模式匹配、替换和分割。
- **字符和数字转换为字符串 (Convert Character and Numbers to Strings)**: `String` 类提供了多个静态 `valueOf` 方法, 可以将字符、字符数组、数字转换为字符串。
- **`StringBuilder` 和 `StringBuffer`**: 用于处理**可变**字符串。 `StringBuilder` 更灵活; `StringBuffer` 类似但方法是**同步**的 (适用于多线程)。
 - 提供多种方法修改字符串: `append()`、`insert()`、`delete()`、`replace()`、`reverse()`、`setCharAt()` 等。

3. Chapter 11: Inheritance and Polymorphism (继承与多态)

3.1. 引言与目标 (Introduction & Objectives)

本章探讨如何使用继承避免代码冗余, 以及多态和动态绑定的概念。目标包括: 定义子类和超类; 使用 `super` 关键字; 理解构造函数链; 重写和重载方法; 了解 `Object` 类; 区分多态和动态绑定; 进行对象类型转换; 使用 `instanceof` 运算符; 使用 `ArrayList` 存储和操作对象; 实现 `MyStack`; 理解 `protected` 修饰符; 以及使用 `final` 修饰符。

3.2. 继承基础 (Inheritance Basics)

- **超类和子类 (Superclasses and Subclasses)**: 超类是更一般的类, 子类是更具体的类。子类可以从超类**继承**属性和方法。
- **子类定义 (Defining a Subclass)**: 子类可以添加新属性、添加新方法, 以及**重写**超类的方法。
- **`final` 修饰符 (The `final` Modifier)**:
 - `final` 类不能被继承。
 - `final` 变量是常量, 值不能改变。
 - `final` 方法不能被子类重写。

3.3. 构造函数与继承 (Constructors & Inheritance)

- **超类构造函数是否被继承? (Are superclass's Constructor Inherited?)**: 不, 超类构造函数不被继承。它们被**显式** (`super(...)`) 或**隐式** (`super()`) 调用。 `super()` 必须是子类构造函数的第一条语句。
- **构造函数链 (Constructor Chaining)**: 构造一个实例会调用继承链上所有超类的构造函数。

3.4. 方法重写与重载 (Method Overriding & Overloading)

- **重写超类中的方法 (Overriding Methods in the Superclass):** 子类可以重写超类中已定义的方法的实现。
 - **注意:** 实例方法只有在其可访问时才能被重写。私有方法不能被重写。
 - **注意:** 静态方法不能被重写, 如果子类重新定义了同名静态方法, 它会**隐藏**超类的方法。
- **调用超类方法 (Calling Superclass Methods):** 使用 `super.methodName()` 调用超类中的方法。
- **重写 vs. 重载 (Overriding vs. Overloading):**
 - **重写 (Overriding):** 子类提供与超类方法**相同签名**的不同实现。
 - **重载 (Overloading):** 在同一个类中定义**同名但参数列表不同**的方法。

3.5. 多态与动态绑定 (Polymorphism & Dynamic Binding)

- **多态 (Polymorphism):** 超类型变量可以引用子类型对象。子类定义的类型是子类型, 超类定义的类型是超类型。
- **动态绑定 (Dynamic Binding):** Java 虚拟机 (JVM) 在运行时动态确定要调用哪个方法实现。当超类型变量引用子类型对象并调用其方法时, JVM 从最具体的类开始搜索实现并调用找到的第一个。
- **方法匹配 vs. 绑定 (Method Matching vs. Binding):**
 - **方法匹配:** 编译器在编译时根据方法签名查找方法。
 - **方法绑定:** JVM 在运行时动态地将方法调用与实际的方法实现关联。
- **泛型编程 (Generic Programming):** 多态允许方法通用地处理各种对象参数。

3.6. 类型转换与操作符 (Casting & Operators)

- **对象类型转换 (Casting Objects):**
 - **隐式类型转换 (Implicit Casting):** 从子类型到超类型是自动的。
 - **显式类型转换 (Explicit Casting):** 从超类型到子类型需要显式转换, 可能失败 (`ClassCastException`)。
- **instanceof 运算符 (The instanceof Operator):** 使用 `instanceof` 运算符测试一个对象是否是某个类的实例, 以在进行向下转型前检查类型安全性。

3.7. 集合框架中的应用 (Applications in Collections Framework)

- **Object 类 (The Object Class):** Java 中所有类都直接或间接继承自 `java.lang.Object`。
`toString()` 方法返回对象的字符串表示, 建议重写。
- **ArrayList 类 (The ArrayList Class):** 提供可变大小的动态数组, 支持添加、删除、获取、设置元素等操作。
 - **泛型类型 (Generic Type):** `ArrayList` 是一个泛型类 (`ArrayList<E>`), 可以指定具体类型。

- **数组和 ArrayList 的区别与相似之处 (Differences and Similarities between Arrays and ArrayList)**: 数组固定大小，随机访问快；ArrayList 动态大小，增删方便，随机访问稍慢。
- **MyStack 类 (The MyStack Classes)**: 基于 ArrayList 实现的栈，提供 push、pop、peek、isEmpty 等栈操作。

3.8. 访问修饰符与 final (Access Modifiers & final)

- **protected 修饰符 (The protected Modifier)**: protected 成员可以被同一包中的类和所有子类访问（无论是否在同一包）。可见性级别：private < default < protected < public 。
- **可访问性摘要 (Accessibility Summary)**: 表格总结了不同修饰符下类成员在同一类、同一包、子类和不同包中的可访问性。

修饰符	同一类	同一包	子类	不同包
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

4. Chapter 12: Exception Handling and Text IO (异常处理和文本I/O)

4.1. 引言与目标 (Introduction & Objectives)

当程序运行时遇到运行时错误，程序会异常终止。如何处理运行时错误，使程序能够继续运行或优雅地终止，是本章的主题。本章目标包括：概述异常和异常处理；探索异常处理的优势；区分异常类型（Error vs. Exception，受检 vs. 非受检）；声明、抛出和捕获异常；理解异常传播；使用 finally 子句；以及定义自定义异常。此外，本章还涵盖文件和文本 I/O：发现文件/目录属性；使用 PrintWriter 和 Scanner 进行文件读写；使用 try-with-resources；以及从 Web 读取数据（Web 爬虫）。

4.2. 异常处理基础 (Exception Handling Basics)

- **异常处理 (Exception Handling)** 是一种简洁地处理程序错误，并分离正常逻辑和异常处理逻辑的方式。相比传统使用 if/else 和返回值处理错误，异常处理代码量更少且逻辑更清晰。

- **异常优势 (Exception Advantages)** 在于允许方法向其调用者抛出异常，而无需在方法内部强制处理或终止程序。
- **异常类型 (Exception Types):**
 - `Throwable` 是所有错误和异常的根类。
 - **Error (系统错误):** 由 JVM 抛出，通常是致命的，不可恢复 (如 `VirtualMachineError`)。
 - **Exception (程序错误或外部情况):** 可捕获和处理。
 - **受检异常 (Checked Exceptions):** `Exception` 的子类，除了 `RuntimeException`。编译器强制检查和处理 (如 `IOException`, `ClassNotFoundException`)。
 - **非受检异常 (Unchecked Exceptions):** `RuntimeException` 及其子类。通常反映编程逻辑错误 (如 `NullPointerException`, `ArithmeticException`, `IndexOutOfBoundsException`, `IllegalArgumentException`)。Java 不强制处理此类异常。

4.3. 异常的声明、抛出与捕获 (Declaring, Throwing, and Catching Exceptions)

- **声明异常 (Declaring Exceptions):** 方法头使用 `throws` 关键字声明可能抛出的受检异常类型。
- **抛出异常 (Throwing Exceptions):** 当程序检测到错误时，创建适当的异常类型实例并使用 `throw` 关键字抛出。例如，`throw new IllegalArgumentException("Radius cannot be negative");`
- **捕获异常 (Catching Exceptions):** 使用 `try-catch` 块捕获和处理异常。一个 `try` 块可以跟随一个或多个 `catch` 块，每个处理不同类型的异常。
- **捕获或声明受检异常 (Catch or Declare Checked Exceptions):** Java 强制处理受检异常。如果方法调用可能抛出受检异常的方法，必须在 `try-catch` 块中捕获它，或者在调用方法中声明抛出该异常。
- **finally 子句 (The finally Clause):** `finally` 块中的语句无论是否发生异常，都会被执行。它常用于清理资源，如关闭文件或网络连接。
- **重新抛出异常 (Rethrowing Exceptions):** 在 `catch` 块中处理部分异常后，可以选择重新抛出异常，以便更高级别的调用者也能处理。

4.4. 异常处理的实践指导 (Practical Guidance for Exception Handling)

- **何时抛出异常 (When to Throw Exceptions):** 如果希望异常由调用者处理，则创建并抛出异常。如果方法内部可以处理异常，则无需抛出。
- **何时使用异常 (When to Use Exceptions):** `try-catch` 块应用于处理**意外错误条件**。不应将其用于处理简单、预期的情况。例如，对 `null` 引用变量的检查，使用 `if (refVar != null)` 比 `try-catch (NullPointerException)` 更高效。

- **自定义异常类 (Defining Custom Exception Classes):** 尽可能使用 API 中已有的异常类。当预定义的类不足以满足需求时，定义自定义异常类。自定义异常类通过扩展 `Exception` 或 `Exception` 的子类来实现。

4.5. 文件I/O基础 (File I/O Basics)

- **File 类 (The File Class):** 提供文件或路径的抽象，处理机器相关的复杂性，提供机器无关的方式。文件名是字符串，目录路径是目录路径。 `File` 对象封装文件属性，但不包含读写数据方法。
- **获取文件属性和操作文件 (Obtaining file properties and manipulating file):** `java.io.File` 类提供文件属性查询（如 `exists()`、`length()`、`isDirectory()`）和文件操作（如 `delete()`、`mkdir()`）方法。
- **文本 I/O (Text I/O):** 通过 `Scanner` 和 `PrintWriter` 类读写文本文件中的字符串和数值。
 - **使用 `PrintWriter` 写入数据 (Writing Data Using `PrintWriter`):** `java.io.PrintWriter` 类用于将数据写入文本文件，支持 `print()` 和 `println()` 系列方法，并支持 `printf` 格式化输出。
 - **使用 `Scanner` 读取数据 (Reading Data Using `Scanner`):** `java.util.Scanner` 类用于从文件或字符串中读取数据，支持 `hasNext()`、`next()`、`nextInt()` 等方法。
 - **try-with-resources :** JDK 7 引入的语法，确保资源（如文件）在块结束时自动关闭，避免资源泄露。

4.6. 二进制I/O与对象序列化 (Binary I/O & Object Serialization)

- **文本文件 vs. 二进制文件 (Text File vs. Binary File):** 二进制文件以位序列存储，由程序读取，处理效率更高，不涉及编码/解码转换。
- **二进制 I/O 类 (Binary I/O Classes):** Java I/O 流体系结构包括 `InputStream` 和 `OutputStream` 的各种子类，如 `FileInputStream` / `FileOutputStream`（用于文件读写），`DataInputStream` / `DataOutputStream`（用于基本类型和字符串的读写），以及 `ObjectInputStream` / `ObjectOutputStream`（用于对象的读写）。
- **缓冲流 (Buffered Streams):** `BufferedInputStream` 和 `BufferedOutputStream` 通过使用缓冲区来加速 I/O 操作。
- **对象序列化 (Object Serialization):**
 - **Serializable 接口 (The Serializable Interface):** 一个标记接口，实现此接口的类允许其对象被序列化（写入对象流）。
 - **transient 关键字 (The transient Keyword):** 标记数据字段，告诉 JVM 在序列化时忽略这些字段，避免 `NotSerializableException`。
 - **序列化数组 (Serializing Arrays):** 如果数组的所有元素都可序列化，则整个数组是可序列化的。
- **随机访问文件 (Random Access Files):** `RandomAccessFile` 类允许在文件的随机位置进行读写操作，而非顺序读写。它包含类似 `DataInputStream` 和 `DataOutputStream` 的读写方法，以及

`seek()` (设置文件指针) 和 `getFilePointer()` (获取文件指针) 等特有方法。

5. Chapter 13: Abstract Classes and Interfaces (抽象类和接口)

5.1. 引言与目标 (Introduction & Objectives)

本章探讨抽象类和接口，它们在定义类共同行为中发挥作用。目标包括：设计和使用抽象类；泛化数值包装类；处理日历；使用接口指定对象共同行为；实现 `Comparable` 和 `Cloneable` 接口；探讨具体类、抽象类和接口异同；设计有理数类；以及遵循类设计指南。

5.2. 抽象类 (Abstract Classes)

- **定义 (Definition)**: 用 `abstract` 关键字修饰的类。抽象类不能被实例化，但可以作为其他类的父类。
- **抽象方法 (Abstract Methods)**: 用 `abstract` 关键字修饰且没有方法体的方法。抽象方法只能在抽象类中声明。
- **特性 (Characteristics)**:
 - **不能创建对象 (Object cannot be created from abstract class)**: 抽象类不能使用 `new` 运算符实例化，但其构造函数可在子类中被调用。
 - **无抽象方法的抽象类 (Abstract class without abstract method)**: 可以定义一个不包含任何抽象方法的抽象类，用作基类，但不能实例化。
 - **超类可以是具体类 (Superclass of abstract class may be concrete)**: 抽象类的超类可以是具体类（如 `Object`）。
 - **具体方法重写为抽象 (Concrete method overridden to be abstract)**: 子类可以将超类中的具体方法重写为抽象方法，此时子类也必须为抽象类。
 - **作为类型使用 (Abstract class as type)**: 抽象类可以作为数据类型使用，例如创建抽象类的数组：`GeometricObject[] geo = new GeometricObject[10];`

5.3. 抽象类应用示例 (Abstract Class Application Examples)

- **抽象 `Number` 类 (Abstract Number Class)**: `java.lang.Number` 是抽象类，是所有数值包装类（`Integer`，`Double` 等）的超类，定义了将数值转换为各种基本数据类型的抽象方法。
- **抽象 `Calendar` 类及其 `GregorianCalendar` 子类 (Abstract Calendar Class & `GregorianCalendar` subclass)**: `java.util.Calendar` 是抽象基类，用于提取日期和时间信

息。 `java.util.GregorianCalendar` 是其具体子类。 `Calendar` 类的 `get(int field)` 方法用于提取日期时间字段。

5.4. 接口 (Interfaces)

- **定义 (Definition):** 一个类似于类的构造，只包含常量和抽象方法。
 - 语法：

```
public interface InterfaceName { constant declarations; abstract method signatures; }
```
 - **省略修饰符 (Omitting Modifiers):** 接口中所有数据字段默认都是 `public final static`，所有方法默认都是 `public abstract`，因此这些修饰符可以省略。
- **特殊类特性 (Special Class Characteristics):** 接口被视为 Java 中的特殊类。不能使用 `new` 运算符实例化，但可以作为数据类型使用。
- **何时使用接口 (When to use an interface?):**
 - 接口用于指定对象的共同行为，即使这些类之间没有直接的继承关系（弱“is-a”关系或“是一种能力”）。
 - 可以用于**规避单继承限制**，实现多重继承的效果。

5.5. 接口应用示例 (Interface Application Examples)

- **Comparable 接口 (The Comparable Interface):** `java.lang.Comparable<E>` 定义了对对象的自然排序，包含 `compareTo(E o)` 方法。许多包装类（如 `Integer`）都实现了 `Comparable`。
- **Cloneable 接口 (The Cloneable Interface):** 一个标记接口，不包含方法。实现此接口的类表明其对象可以被克隆（通过 `Object.clone()` 方法）。
 - **浅拷贝 vs. 深拷贝 (Shallow vs. Deep Copy):** 浅拷贝只复制引用，深拷贝则递归复制所有对象以完全独立。

5.6. 接口与抽象类的对比 (Interfaces vs. Abstract Classes)

特性	抽象类 (Abstract class)	接口 (Interface)
变量	无限制（可以有各种类型）	必须是 <code>public static final</code> 常量
构造函数	被子类通过构造函数链调用；不能直接实例化	没有构造函数；不能实例化
方法	无限制（可以有具体方法、抽象方法）	必须是 <code>public abstract</code> 实例方法

- **共同点与区别:** 所有类都继承自 `Object`，但接口没有单一的根。接口可以作为数据类型并进行类型转换。

- **冲突接口 (Conflict Interfaces)**: 一个类可能实现两个存在冲突信息的接口 (如同名常量或同名但返回类型不同的方法), 这会被编译器检测到。

5.7. 类设计指南 (Class Design Guidelines)

- **内聚性 (Coherence)**: 一个类应描述一个单一实体, 所有操作应逻辑地支持一个内聚目的。
- **分离职责 (Separating Responsibilities)**: 职责过多的实体可拆分为多个类。
- **重用性**: 设计类时应考虑其不同组合、顺序和环境下的重用性。属性应可按任意顺序设置, 方法应独立于其调用顺序。
- **可见性修饰符 (Visibility Modifiers)**: `private` (隐藏数据, 通过 `get / set` 访问); `protected` (供扩展者使用); `public` (供用户使用)。
- **static 修饰符 (The static Modifier)**: 由类的所有实例共享的属性应声明为 `static` 属性。
- 提供**公共无参构造函数**, 并尽可能重写 `Object` 类中的 `equals` 和 `toString` 方法。
- 遵循标准 Java 编程风格和命名约定。选择信息丰富的名称。始终将数据声明放在构造函数之前, 构造函数放在方法之前。始终提供构造函数并初始化变量, 以避免编程错误。

6. Chapter 14: JavaFX Basics (JavaFX 基础)

6.1. 引言与目标 (Introduction & Objectives)

JavaFX 是一个用于开发 Java GUI 程序的新框架, 展示了面向对象原则的优秀应用。本章旨在介绍 JavaFX 编程基础, 并演示如何用 JavaFX 实现 OOP。具体而言, 本章将介绍 JavaFX 框架以及 GUI 组件及其关系。目标包括: 区分 JavaFX、Swing 和 AWT; 理解舞台、场景和节点的关系; 使用窗格、UI 控件和形状创建 UI; 使用绑定属性; 使用通用属性 `style` 和 `rotate`; 使用 `Color` 和 `Font`; 使用 `Image` 和 `ImageView`; 使用各种布局窗格; 使用 `Text` 和其他形状; 以及开发 `ClockPane`。

6.2. JavaFX 概述 (JavaFX Overview)

- **JavaFX vs Swing and AWT (JavaFX 与 Swing 和 AWT 对比)**: JavaFX 旨在取代 Swing 和 AWT, 用于开发丰富的互联网应用程序。AWT 是早期 GUI 库, Swing 是其增强版, JavaFX 是 Java 8 引入的全新 GUI 平台。
- **JavaFX 项目创建 (Creating JavaFX Projects)**: 可以通过 Maven archetype 或 VSCode 插件 (如 Extension Pack for Java) 来创建。
- **基本结构 (Basic Structure)**: JavaFX 应用程序的基本结构包括 `Application` 类 (入口点, 需重写 `start(Stage)` 方法)、`Stage` (顶级窗口容器)、`Scene` (舞台中的内容容器) 和 `Nodes` (场景中的图形元素)。

6.3. JavaFX UI 元素 (JavaFX UI Elements)

- **节点层次 (Nodes Hierarchy)**: 所有可见元素（包括 UI 控件、形状等）的基类是 `Node`。`Parent` 是可以包含其他节点的节点（如 `Pane` 和 `Control`）。
- **UI 控件 (UI Controls)**: 用户界面交互元素，如 `Label`、`Button`、`CheckBox`、`RadioButton`、`TextField`、`TextArea`、`ComboBox`、`ListView`、`ScrollBar`、`Slider`。
 - **Labeled**: 是显示短文本、节点或两者皆有的显示区域的基类，定义了通用属性如 `text`、`alignment`、`graphic` 等。`Label`、`ButtonBase` (基类，包含 `Button`、`CheckBox`、`RadioButton`) 都继承自 `Labeled`。
 - **TextField 和 TextArea**: 用于文本输入。
 - **ComboBox 和 ListView**: 用于从列表中选择项目。
 - **ScrollBar 和 Slider**: 用于选择范围值。
- **形状 (Shapes)**: JavaFX 提供多种形状类，用于绘制图形，如 `Text`、`Line`、`Rectangle`、`Circle`、`Ellipse`、`Arc`、`Polygon` 和 `Polyline`。

6.4. 布局管理 (Layout Management)

- **布局窗格 (Layout Panes)**: JavaFX 提供多种窗格来组织容器中的节点。
 - **Pane**: 布局窗格的基类。
 - **StackPane**: 节点堆叠居中。
 - **FlowPane**: 节点按行或列排列。
 - **GridPane**: 节点放置在二维网格中。
 - **BorderPane**: 节点放置在顶部、右侧、底部、左侧和中心区域。
 - **HBox**: 节点在单行中放置。
 - **VBox**: 节点在单列中放置。

6.5. 属性与样式 (Properties & Styling)

- **绑定属性 (Binding Properties)**: 允许一个目标属性与源属性绑定，源属性变化时目标属性自动更新。支持单向和双向绑定。
- **通用属性和方法 (Common Properties and Methods)**: 节点具有 `style` (设置 CSS 样式) 和 `rotate` (旋转) 等通用属性。
- **Color 类 (The Color Class)**: 用于表示颜色，包含 `RGB` 和透明度属性。
- **Font 类 (The Font Class)**: 用于管理字体，包含 `size`、`name`、`family` 等属性。

6.6. 图像与多媒体 (Images & Multimedia)

- **Image 类 (The Image Class)**: 用于加载图像。

- **ImageView 类 (The ImageView Class)**: 是 `Node` 的子类, 用于显示 `Image` 对象。
- **Media 类 (Media Class)**: 用于获取媒体源。
- **MediaPlayer 类 (MediaPlayer Class)**: 用于播放和控制媒体。
- **MediaView 类 (MediaView Class)**: 是 `Node` 的子类, 用于显示媒体内容。

7. Chapter 15: Event-Driven Programming and Animations (事件驱动编程和动画)

7.1. 引言与目标 (Introduction & Objectives)

本章探讨事件驱动编程, 它是 GUI 程序响应用户交互的基础。目标包括: 理解事件驱动编程; 描述事件、事件源和事件类; 定义处理类并注册监听器; 使用内部类、匿名内部类和 Lambda 表达式处理事件; 开发 GUI 应用程序 (如贷款计算器); 处理鼠标和键盘事件; 使用可观察对象监听值变化; 以及创建各种动画 (如 `PathTransition`、`FadeTransition`、`Timeline`) 和模拟弹跳球。

7.2. 事件驱动编程核心 (Core Event-Driven Programming)

- **过程式 vs. 事件驱动编程 (Procedural vs. Event-Driven Programming)**: 过程式编程按顺序执行代码; 事件驱动编程在事件激活时执行代码, 是 GUI 交互的基础。
- **GUI 事件处理 (Handling GUI Events)**:
 - **事件源 (Source object)**: 生成事件的对象 (例如按钮)。
 - **监听器 (Listener object)**: 包含处理事件的方法。
 - **事件 (Event)**: 程序收到的一种信号, 表明发生了某事 (如鼠标点击、键盘按键)。
 - **委托模型 (Delegation Model)**: 事件源将事件处理委托给监听器。事件源注册监听器, 事件发生时调用监听器的 `handle` 方法。

7.3. 事件类与信息 (Event Classes & Information)

- **事件类 (Event Classes)**: JavaFX 事件类位于 `javafx.event` 包中, 形成层次结构 (如 `ActionEvent`、`MouseEvent`、`KeyEvent`)。
- **事件信息 (Event Information)**: 事件对象包含与事件相关的所有属性, 可以使用 `EventObject` 的 `getSource()` 方法识别事件源。
- **常用用户操作与处理程序 (Selected User Actions and Handlers)**: 表格列举了常见的 UI 控件 (如 `Button`、`TextField`、`RadioButton`、`CheckBox`、`ComboBox`) 及其对应的事件类型 (`ActionEvent` 或 `MouseEvent` / `KeyEvent`) 和事件注册方法 (如 `setOnAction`、`setOnMousePressed`、`setOnKeyPressed`)。

7.4. 事件处理方式 (Event Handling Approaches)

- **内部类监听器 (Inner Class Listeners)**: 当监听器类只用于特定 GUI 组件时不共享时, 适合定义为外部类的内部类。内部类可以访问外部类的数据和方法。
- **匿名内部类 (Anonymous Inner Classes)**: 没有名称的内部类, 直接在创建实例时声明。必须扩展超类或实现接口, 但不能有显式的 `extends / implements` 子句。编译后名为 `OuterClassName$n.class`。
- **Lambda 表达式 (Lambda Expressions)**: Java 8 引入, 简化了匿名内部类, 特别是针对**单抽象方法接口 (SAM)**。基本语法为 `(parameters) -> expression` 或 `(parameters) -> { statements; }`。

7.5. JavaFX 动画 (JavaFX Animations)

- **Animation 类 (Animation Class)**: JavaFX 动画的核心功能基类。
- **PathTransition (路径过渡)**: 动画化节点沿着给定路径移动。
- **FadeTransition (淡入淡出过渡)**: 动画化节点透明度的变化。
- **Timeline (时间轴)**: 通过一个或多个 `KeyFrame` 程序化地创建动画, 继承自 `Animation`。

8. Chapter 16: JavaFX UI Controls and Multimedia (JavaFX UI 控件和多媒体)

8.1. 引言与目标 (Introduction & Objectives)

GUI 使系统用户友好且易于使用, 创建 GUI 需要创造力和对组件工作原理的理解. Java 中的 GUI 组件灵活多变. 本章详细介绍常用 GUI 组件. 目标包括: 使用各种 UI 控件创建 GUI; 使用 `Label` 及其属性; 使用 `Button` 及其 `setOnAction`; 使用 `CheckBox` 和 `RadioButton` (配合 `ToggleGroup`); 使用 `TextField` 和 `PasswordField` 输入数据; 使用 `TextArea` 输入多行数据; 使用 `ComboBox` 单选; 使用 `ListView` 单选/多选; 使用 `ScrollBar` 和 `Slider` 选择范围; 开发 Tic-Tac-Toe 游戏; 使用 `Media`、`MediaPlayer` 和 `MediaView` 播放视频和音频; 以及开发显示国旗和播放国歌的案例研究。

8.2. UI 控件基础 (UI Control Basics)

- **常用 UI 控件 (Frequently Used UI Controls)**: JavaFX 中常用的 UI 控件包括: `Label`, `Button`, `CheckBox`, `RadioButton`, `TextField`, `PasswordField`, `TextArea`, `ComboBox`, `ListView`, `ScrollBar`, `Slider`. 这些控件通常继承自 `Control` 或 `Labeled`。
- **Labeled (带标签的控件)**: 是显示短文本、节点或两者皆有的显示区域的基类. 它定义了通用属性如 `text`、`alignment`、`graphic` 等. `Label`、`ButtonBase` (基类, 包含

Button、CheckBox、RadioButton) 都继承自 Labeled。

8.3. 文本输入与选择控件 (Text Input & Selection Controls)

- **Label (标签)**: 定义标签, 可创建空标签、带文本的标签、或带文本和图形的标签。
- **ButtonBase and Button (按钮基类和按钮)**: ButtonBase 是触发动作事件的控件基类. Button 是触发点击事件的按钮. 共同特性是 onAction (动作事件处理器)。
- **CheckBox (复选框)**: 用于用户进行选择. 继承 ButtonBase 和 Labeled 的属性. 主要属性为 selected (是否选中)。
- **RadioButton (单选按钮)**: 允许用户从一组选项中选择单个项目. 继承 ToggleButton 属性. 主要属性是 selected 和 toggleGroup (所属的单选组)。
- **TextField (文本字段)**: 用于输入或显示单行字符串. 它是 TextInputControl 的子类. 主要属性有 text (文本内容)、editable (是否可编辑)、alignment (文本对齐)。
- **TextArea (文本区域)**: 允许用户输入多行文本. 主要属性有 prefColumnCount (首选列数)、prefRowCount (首选行数)、wrapText (是否换行)。
- **ComboBox (组合框)**: 允许用户从列表选择一个项目, 也称为选择列表或下拉列表. 主要属性有 value (选中的值)、editable (是否可编辑)、items (列表项)。
- **ListView (列表视图)**: 允许用户选择单个或多个值. 主要属性有 items (列表项)、orientation (方向) 和 selectionModel (选择模型)。

8.4. 数值选择控件 (Numeric Selection Controls)

- **ScrollBar (滚动条)**: 允许用户从一个范围中选择一个值, 有水平和垂直两种样式. 属性包括 blockIncrement、max、min、unitIncrement、value、visibleAmount、orientation。
- **Slider (滑块)**: 类似于 ScrollBar, 但具有更多属性, 并可以以多种形式出现. 增加了 majorTickUnit (主刻度单位)、minorTickCount (次刻度数量)、showTickLabels (显示刻度标签) 和 showTickMarks (显示刻度标记)。

8.5. 多媒体与案例研究 (Multimedia & Case Studies)

- **Media (媒体)**: Media 类用于获取媒体源。
- **MediaPlayer (媒体播放器)**: MediaPlayer 类用于播放和控制媒体. 属性如 autoPlay、currentCount、cycleCount、mute、volume、totalDuration。
- **MediaView (媒体视图)**: MediaView 类是 Node 的子类, 用于显示媒体内容。

9. Chapter 17: Binary I/O (二进制 I/O)

9.1. 引言与目标 (Introduction & Objectives)

本章探讨二进制 I/O 的概念和应用。数据存储在文本文件中是人类可读的形式，而二进制文件是程序可读的形式，处理效率更高。目标包括：了解 Java 中 I/O 处理方式；区分文本 I/O 和二进制 I/O；使用 `FileInputStream` 和 `FileOutputStream` 读写字节；使用 `DataInputStream` 和 `DataOutputStream` 读写基本类型值和字符串；使用 `ObjectOutputStream` 和 `ObjectInputStream` 存储和恢复对象（序列化）；实现 `Serializable` 接口；序列化数组；以及使用 `RandomAccessFile` 读写随机位置的文件。

9.2. Java I/O 概述 (Java I/O Overview)

- **Java I/O 处理方式 (How is I/O Handled in Java?):** `File` 对象封装文件属性，但不包含读写数据方法。读写数据需要使用相应的 Java I/O 类创建对象。
- **文本文件 vs. 二进制文件 (Text File vs. Binary File):** 二进制文件以位序列存储，由程序读取，处理效率更高，不涉及编码/解码转换。

9.3. 二进制 I/O 流 (Binary I/O Streams)

- **二进制 I/O 类层次结构 (Binary I/O Classes Hierarchy):**
 - `Object` -> `Throwable` -> `InputStream` (输入流基类) / `OutputStream` (输出流基类)。
 - 具体流类包括：`FileInputStream` / `FileOutputStream` (用于文件读写),
`DataInputStream` / `DataOutputStream` (用于基本类型和字符串的读写),
`ObjectInputStream` / `ObjectOutputStream` (用于对象的读写),
`BufferedInputStream` / `BufferedOutputStream` (用于缓冲，加速 I/O)。
- **`InputStream` 和 `OutputStream`:** 分别是所有输入/输出流的抽象基类。提供 `read()` / `write()` 等基本字节操作。
- **文件流 (`FileInputStream` / `FileOutputStream`):** 将二进制 I/O 流与外部文件关联。
`FileOutputStream` 可指定追加模式。
- **过滤流 (`FilterInputStream` / `FilterOutputStream`):** 将原始字节流包装起来，以提供更高级的读写功能，如读写整数、双精度浮点数或字符串。
- **数据流 (`DataInputStream` / `DataOutputStream`):** 是过滤流的子类，用于读写 Java 基本数据类型和 UTF-8 编码的字符串。
 - **注意:** 读写数据时必须保持相同的顺序和格式。
- **缓冲流 (`BufferedInputStream` / `BufferedOutputStream`):** 通过使用内部缓冲区来加速 I/O 操作。

9.4. 对象序列化 (Object Serialization)

- **Serializable 接口 (The Serializable Interface)**: 一个标记接口, 实现此接口的类允许其对象被序列化 (写入对象流)。
- **transient 关键字 (The transient Keyword)**: 标记数据字段, 告诉 JVM 在序列化时忽略这些字段. 静态变量也不会被序列化。
- **对象流 (ObjectInputStream / ObjectOutputStream)**: 用于存储和恢复对象. `readObject()` 读取对象, `writeObject()` 写入对象。
- **序列化数组 (Serializing Arrays)**: 如果数组的所有元素都可序列化, 则整个数组是可序列化的。

9.5. 随机访问文件 (Random Access Files)

- `RandomAccessFile` 类允许在文件的随机位置进行读写操作, 而非顺序读写。
- 它包含类似 `DataInputStream` 和 `DataOutputStream` 的读写方法, 以及 `seek()` (设置文件指针) 和 `getFilePointer()` (获取文件指针) 等特有方法。
- **文件指针 (File Pointer)**: 一个特殊的标记, 指示文件读写操作的当前位置。
- **构造函数**: `RandomAccessFile(file: File, mode: String)` 或 `RandomAccessFile(name: String, mode: String)`, 模式可以是 "r" (只读) 或 "rw" (读写)。

10. Chapter 19: Generics (泛型)

10.1. 引言与目标 (Introduction & Objectives)

泛型是 Java 中参数化类型的能力, 能显著提高代码的可靠性和可读性. 本章目标包括: 了解泛型的好处; 使用和声明泛型类、接口和方法; 理解有界泛型类型和通配符; 认识类型擦除及其限制; 以及设计泛型矩阵类。

10.2. 泛型基础 (Generics Basics)

- **什么是泛型? (What is Generics?)**: 泛型是参数化类型的能力, 允许定义带有泛型类型 (在编译时替换为具体类型) 的类或方法。
- **为什么使用泛型? (Why Generics?)**: 主要好处是在编译时而非运行时检测到错误. 如果尝试使用不兼容的对象, 会发生编译错误。
- **原始类型与向后兼容性 (Raw Type and Backward Compatibility)**: 不指定类型参数的泛型类型称为原始类型 (Raw Type). 原始类型是为了向后兼容旧版代码而存在的。
 - **原始类型不安全 (Raw Type is Unsafe)**: 使用原始类型可能导致运行时 `ClassCastException` 错误, 因为编译器无法进行严格的类型检查. 应避免使用不安全的原始类型。

10.3. 泛型类与接口 (Generic Classes & Interfaces)

- **声明泛型类和接口 (Declaring Generic Classes and Interfaces)**: 通过在类名或接口名后加上 `<E>` (或其他类型参数名) 来声明。
- **泛型实例化 (Generic Instantiation)**: 在创建泛型类的对象时指定具体类型, 例如 `ArrayList<String>`。这提高了可靠性并实现了编译时错误检查。
- **ArrayList 中的泛型 (Generic ArrayList in JDK 1.5)**: JDK 1.5 引入了泛型 `ArrayList<E>`, 使得方法参数和返回值都是类型参数 `E`, 无需手动类型转换, 简化了编程。
- **有界泛型类型 (Bounded Generic Type)**: 泛型类型参数可以被限定在一个特定的类型或其子类型中, 例如 `<E extends GeometricObject>`。这增强了类型安全性。

10.4. 泛型方法与通配符 (Generic Methods & Wildcards)

- **泛型方法 (Generic Methods)**: 方法也可以声明为泛型, 允许它们处理不同类型的参数, 提供更好的类型安全和代码重用。
- **通配符 (Wildcards)**: `?` 用于表示未知类型, 增加泛型代码的灵活性。
 - `?`: **无界通配符** (unbounded wildcard), 表示任何类型。
 - `? extends T`: **有界通配符** (bounded wildcard), 表示 `T` 类型或其子类型。
 - `? super T`: **下限通配符** (lower bound wildcard), 表示 `T` 类型或其超类型。

10.5. 类型擦除与泛型限制 (Type Erasure & Generics Restrictions)

- **类型擦除 (Type Erasure)**: 泛型通过类型擦除实现。编译器在编译时使用泛型类型信息, 但在字节码中会擦除它。这使得泛型代码能与旧版代码向后兼容。
- **编译时检查 (Compile Time Checking)**: 编译器在编译时检查泛型是否正确使用, 并插入必要的类型转换。
- **重要事实 (Important Facts)**: 泛型类被其所有实例共享, 无论其实际泛型类型如何。例如, `GenericStack<String>` 和 `GenericStack<Integer>` 在运行时都对应同一个 `GenericStack` 类。
- **泛型限制 (Restrictions on Generics)**:
 - i. 不能创建泛型类型的实例 (例如 `new E()` 不允许)。
 - ii. 不允许创建泛型数组 (例如 `new E[100]` 不允许)。
 - iii. 类的泛型类型参数不允许在静态上下文中使用。
 - iv. 异常类不能是泛型。

11. Chapter 20: Lists, Stacks, Queues, and Priority Queues (列表、栈、队列和优先队列)

11.1. 引言与目标 (Introduction & Objectives)

本章深入探讨 Java 集合框架中的数据结构，包括列表、栈、队列和优先队列。目标包括：探索 Java Collections Framework 层次结构；使用 Collection 接口操作集合；使用 Iterator 和 for-each 循环遍历元素；理解 ArrayList 和 LinkedList 的选择；使用 Comparable 和 Comparator 比较元素；使用 Collections 工具类；开发集合相关的案例研究；以及理解栈和队列的特性与应用。

11.2. 数据结构与集合框架 (Data Structures & Collections Framework)

- **数据结构 (What is Data Structure?)**: 按照某种方式组织的数据集合，支持访问和操作数据的功能。
- **Java 集合框架层次结构 (Java Collection Framework hierarchy)**: 集合是存储一组对象的容器，支持列表 (lists)、集 (sets) 和映射 (maps)。Set 和 List 是 Collection 的子接口。
 - **Collection 接口 (The Collection Interface)**: 提供操作对象集合的通用方法，如 add()、remove()、contains()、size() 等。

11.3. 列表 (Lists)

- **List 接口 (The List Interface)**: 存储元素按顺序排列，允许用户指定元素存储位置，并可以通过索引访问元素。
- **ListIterator (列表迭代器)**: java.util.ListIterator<E> 扩展了 Iterator<E>，提供了双向遍历列表和修改列表的功能。
- **ArrayList VS. LinkedList**:
 - **ArrayList**: 基于数组实现，支持通过索引进行随机访问效率高。插入或删除非末尾元素效率低。
 - **LinkedList**: 基于链表实现，支持在任意位置插入或删除元素效率高。随机访问效率低。

11.4. 比较器 (Comparators)

- **Comparator 接口 (The Comparator Interface)**: 用于定义比较两个对象的方法，当元素不是 Comparable 类型或需要不同排序规则时使用。compare(Object element1, Object element2) 返回负数、零或正数。
- **Collections 工具类 (The Collections Class)**: 包含各种静态方法，用于操作集合和映射、创建同步集合、创建只读集合，以及进行排序、搜索、洗牌、查找最大/最小元素等操作。

11.5. 栈 (Stacks)

- **Vector 类 (The Vector Class)**: 在 Java 2 之前已支持，现在是集合框架的一部分. 与 `ArrayList` 类似，但其方法是**同步的**。
- **Stack 类 (The Stack Class)**: 表示一种**后进先出 (LIFO)** 的堆栈对象. 元素只能从堆栈顶部访问，提供 `push`、`pop`、`peek`、`empty`、`search` 等操作。
- **案例研究：表达式求值 (Evaluating Expressions)**: 栈可用于评估表达式，涉及扫描表达式并使用操作数栈和操作符栈进行处理。

11.6. 队列与优先队列 (Queues & Priority Queues)

- **队列 (Queues)**: 一种**先进先出 (FIFO)** 的数据结构. 元素在队列末尾追加，从队列开头删除。
- **Queue 接口 (The Queue Interface)**: 定义了队列操作，如 `offer()` (插入)、`poll()` (检索并删除)、`peek()` (检索但不删除)。
- **LinkedList 作为队列 (Using LinkedList for Queue)**: `java.util.LinkedList<E>` 实现了 `List<E>`、`Queue<E>` 和 `Deque<E>` 接口，因此可以作为队列使用。
- **优先队列 (Priority Queues)**: 元素被分配优先级，优先级最高的元素首先被删除。
- **PriorityQueue 类 (The PriorityQueue Class)**: 实现了 `Queue<E>` 接口，提供优先队列功能。